

PART 1

Introduction & Project Overview

FinVault – MERN Stack Virtual Stock Trading Platform

DOCUMENT INFORMATION

Project Title	FinVault – Virtual Stock Trading Platform
Document Phase	Phase 1: Project Documentation
Part	Part 1 – Introduction & Project Overview
Author	Ganesh Basode
Affiliation	Department of Computer Science & Engineering
Institution	G. Pullaiah College of Engineering and Technology (GPCET)
Date	July 2026
Status	Final Version

1. Introduction & Overview

FinVault is a state-of-the-art virtual stock trading platform built using the MERN (MongoDB, Express.js, React.js, Node.js) stack. The platform serves as an interactive educational environment, allowing users to experience stock trading in a simulated sandbox using mock virtual balance. This enables students, beginners, and stock market enthusiasts to understand trading principles, manage dynamic portfolios, and review transactions without exposing themselves to real financial risks.

1.1 Core Objectives

- **Risk-Free Market Education:** Build a safe sandbox environment where users can play, trade, and learn.
- **Technical Competence:** Demonstrate a modern, decoupled full-stack architecture utilizing industry-standard REST APIs and JWT security.
- **Dynamic Visualization:** Provide intuitive UI components, including historical price charts, allowing users to make data-driven simulated investment decisions.
- **Role-Based Workflows:** Differentiate between investor actions and administrator actions to simulate real-world platform controls.

1.2 Target Audience

FinVault is tailored for engineering students, financial literacy workshops, and absolute beginners. By removing the stress of financial losses, users can freely test technical trading patterns, asset allocation concepts, and transaction analysis in a controlled workspace.

2. Key System Features

2.1 User Features

- **User Authentication:** Safe signup and login backed by JWT verification and encrypted credentials.
- **Profile Dashboard:** User profile cards with dynamic mock wallet balances (starting at \$100,000) and profile picture uploads.
- **Stock Browsing & Search:** Interactive list of available stock items with symbol and name-based search filters.
- **Trading Engine:** Support for immediate BUY and SELL actions. Wallet balances and stock inventory update in real-time.
- **Watchlist:** Personal watchlist catalog allowing users to pin specific stocks for regular tracking.
- **Portfolio Tracker:** Summarized table indicating holding stocks, accumulated quantities, average buy price, current price, and net returns.
- **Transaction Log:** History log detailing every transaction (BUY/SELL) including date, quantity, execution price, and total amount.

2.2 Administrator Features

- **Admin Dashboard:** Displays critical KPIs such as total users, stock count, and aggregate transactions.

- **Catalog Operations (CRUD):** Add new stock listings, edit current prices or quantity supplies, and delete inactive listings.
- **Transaction Audit Feed:** Global feed showing trading logs across all users for monitoring.

3. Technology Stack

The platform is engineered using decoupled front-end and back-end modules to support scalability and standard microservices patterns.

3.1 Frontend Stack

- **React.js (Vite):** UI rendering, Virtual DOM management, and state-driven interfaces.
- **React Router DOM (v7):** Client-side routing to manage dashboard sub-pages.
- **Axios:** Asynchronous client for HTTP operations targeting the REST backend.
- **Chart.js:** Canvas-driven historical line plots for stock trends.
- **Vanilla CSS:** High-performance custom layout and styling.

3.2 Backend Stack

- **Node.js & Express.js:** Modular backend API framework handling JSON requests.
- **Mongoose:** Object Document Mapping (ODM) layer for database interactions.
- **bcrypt:** Secure hashing function for encryption of user credentials.
- **Multer:** Streamlined file upload middleware for handling profile pictures.

3.3 Database & Tools

- **MongoDB Atlas:** Managed cloud database cluster storing platform collections.
- **Thunder Client:** Testing tool for validating server response structures and edge cases.